

Are Long-Running Agents Making Progress?

A Longitudinal Study of Artifact Progress, Repeated Mutation, and Continuity

Anonymous submission

Abstract

Long-running agents can modify a workspace for days across many session boundaries. When they stop, action counts, commits, and final artifacts do not reveal whether activity accumulated into durable and validated progress or repeatedly overwrote, abandoned, and rediscovered the same work. Existing trajectory studies characterize actions within benchmark attempts, while studies of long-lived agents focus on task outcomes or internal memory degradation. We instead make the persistent workspace the longitudinal unit of analysis. We introduce a source-linked projection that connects native Claude, Codex, and Gemini actions to stable artifact identities, lifecycle effects, event time, and session boundaries without introducing semantic labels or aligning the timeline to Git commits. We use it to study six real software and autonomous- research projects through introduced-artifact persistence, later reuse, validation association, repeated mutation, workspace-activity migration, and cross-session continuity. Across 2,049 sessions and 206,249 Tool actions, 89.8–97.3% of eligible mutations are later revisited, yet action volume does not reproduce the cross-project reuse ordering ($\rho = 0.09$). Repeat-observed episodes account for 71.8–91.8% of mutation episodes, with 41.9–86.7% of episodes concentrated in the most-mutated 10% of identities. Source coverage is itself consequential: recognized successful validation appears in only 3/6 cases, and skill exposure cannot be identified. A preregistered readiness audit further shows that the frozen corpus lacks the native-prefix and cutoff workspace contracts needed for a fair matched baseline comparison; we report those conditions as N/A rather than manufacturing a superiority result.

Introduction

An increasingly common workflow gives an agent a broad idea, leaves it to work inside one repository for two or three days, and inspects the result later. The returned workspace may contain hundreds of tool calls, dozens of commits, and substantial code, prose, tests, and experimental output. This volume looks productive but does not answer the question the user actually has: *did the work converge into durable, validated progress?*

Final state and commit history are necessary but lossy views of that question. They omit reads, failed validations, transient

artifacts, overwritten attempts, and the cost of re-establishing context after a session boundary. Aggregate counts retain activity volume but erase which artifacts actions concerned and whether later work reused or replaced them. A raw log retains detail, yet its session-local linear order does not directly expose a workspace lineage that continues after one context disappears. Context management can rapidly weaken earlier plans (Mehta and Datta 2026), and reliability can change over long chains of sessions (Zhu et al. 2026).

Fine-grained trajectory analysis is not new. Prior work mines action sequences and repeated behaviors in software-engineering agents (Bouzenia and Pradel 2025), measures validation and context-gathering behavior under task and model confounds (Mehtiyev and Assunção 2026), and shows that agents may reach a correct intermediate state and later overwrite it (Kim et al. 2026). ProcGrep represents trajectories as action procedures and supports deterministic pattern queries and behavioral fingerprints (Oderinwale 2026). Persistent file workspaces and multi-session tasks are also established system mechanisms and settings (Zhou et al. 2026). These results rule out generic claims about first trajectory analysis, edit loops, validation gaps, fingerprints, or persistent workspaces.

The remaining gap is longitudinal and artifact centered. A benchmark attempt usually ends when one patch is scored; a persistent project continues while code, papers, datasets, results, and documentation acquire histories of their own. We ask how action volume relates to three observable dimensions: *introduced-artifact persistence*, whether an observably introduced artifact persists; *reuse*, whether later work returns to it; and *validation association*, whether an adapter-recognized successful validation precedes the artifact’s next mutation or deletion. We keep these dimensions separate instead of combining them with arbitrary weights. Their conjunction is useful shorthand for durable, reused, and validation-associated artifact progress, not a scalar quality score.

We build this study directly on native Agent records. The existing `agent-session` adapters parse Claude, Codex, and Gemini. A thin repository projection retains all admitted Tool actions, resolves qualified file effects, preserves explicit rename lineage and session identity, and orders events by Agent action time. Git contributes final tracked state and optional content-survival evidence, but commits never define the time axis. The same facts drive both quantitative analyses

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

and the auxiliary AGENT NEBULA replay; visual layout is not a statistical measurement.

This paper makes three contributions:

1. a longitudinal empirical characterization of how Agent activity, artifact persistence, reuse, validation, repeated mutation, workspace activity, and session continuity interact across six real coding and autonomous-research projects;
2. a deterministic, source-linked measurement method that preserves stable artifact identity and cross-session lineage without another general event IR or generated semantic gold; and
3. a reproducible source-coverage and benchmark-readiness audit that separates measured results, coverage-only evidence, and N/A conditions instead of treating missing trajectory evidence as observed absence.

Problem Setting

Persistent Work Rather Than Isolated Sessions

Let a workspace W persist while sessions $S = (s_1, \dots, s_m)$ execute native Tool actions $A = (a_1, \dots, a_n)$. Each action may read, create, mutate, rename, or delete zero or more artifacts. Sessions partition model context; they do not reset W or terminate an artifact’s history. Our unit of analysis is one repository lineage over the complete observation interval, with project-level cases kept separate.

Action time is ordered by native timestamp and stable source identifier. A commit can occur between actions and can anchor a final-state query, but it does not replace the action order. This distinction is important because an agent can explore, fail, create temporary files, and revert changes before any commit.

Observable Progress Without Semantic Gold

We do not infer intent, failure cause, reflection, or waste from file motion. Instead, we measure source-verifiable process facts:

- **Introduced-artifact persistence:** final existence or tracked state for artifacts observably introduced during the trace; existing-file writes have unknown content durability without independent evidence;
- **Reuse:** a later source-linked access to the same artifact lineage and its event, time, and session distance; and
- **Validation association:** distance from mutation to the next adapter-recognized successful test, check, or build action before that artifact’s next mutation or deletion.

Recognized validation proves only that the adapter observed a successful command, not that it covered a particular mutation; unrecognized commands and unknown statuses remain coverage gaps. A file that is never revisited is an observed orphaned artifact, not automatically useless. Repeated tests or edits are process patterns, not automatically waste. Skill and harness analyses therefore report temporal association, never causality, in the observational corpus.

Figure 1 makes the measurement relationships explicit. The study follows Agent action time through session boundaries,

keeps artifact histories distinct, and measures survival, later reuse, validation distance, and observable re-grounding. The figure is a definition schematic rather than an empirical result; the corresponding result plots are generated from the six-project source rows in Section .

Workspace-Centered Measurement

Source Selection and Projection

We admit a complete native session when its project directory, working directory or worktree root, or Git remote identifies the target repository. All Tool actions in an admitted session remain on the timeline, including searches, shell commands, failed validations, and calls without a resolved file path. A global path scan is reported separately because matching only repository-related lines loses the surrounding no-file actions.

For each admitted action, the projection retains

$$a_i = (t_i, id_i, s_i, v_i, u_i, e_i, q_i), \quad (1)$$

where t_i is native time, id_i the stable source ID, s_i the session, v_i the vendor, u_i the Tool/category, e_i the adapter-derived command effect, and q_i the reported status. It attaches zero or more effects

$$E_i = \{(w, f, o, f')\}, \quad (2)$$

$$o \in \{\text{read, write, create, rename, delete}\}.$$

Here w is a hashed worktree identifier, f is a repository-relative artifact, and f' is populated only by an observed rename. Failed calls remain actions but emit no confirmed-success effect. Directory arguments are weak scope evidence rather than fabricated reads of every descendant. Unknown paths, effects, and status remain explicit coverage gaps.

Artifact and Validation Derivations

Artifact identity is keyed by hashed worktree ID and relative path. Tool-level workdir takes precedence over session cwd. An explicit same-worktree rename preserves identity and its birth state; only a confirmed-success create makes an identity eligible as an introduction inside the observation. Unknown-status creates retain a separate birth state. Delete followed by create begins a new identity. For each confirmed non-delete mutation, we locate later reuse and adapter-recognized successful validation before the same artifact’s next mutation/delete. Supersession is a competing outcome and only observation end is right-censored. Final existence and tracked state are queried in the corresponding worktree.

We report complete survival and distance curves or declared sensitivity ranges; no fixed 24-event attention window enters the research measurement. Visual attention decay and force layout remain presentation-only choices.

Matched-Comparison Prerequisites

A valid tool comparison would stratify questions into action-only, artifact-linked, cross-session, and final-state facts. Every answer would cite source IDs or abstain, and no method under comparison could label its own answer. Final State and Counts would be lower-information controls, ProcGrep

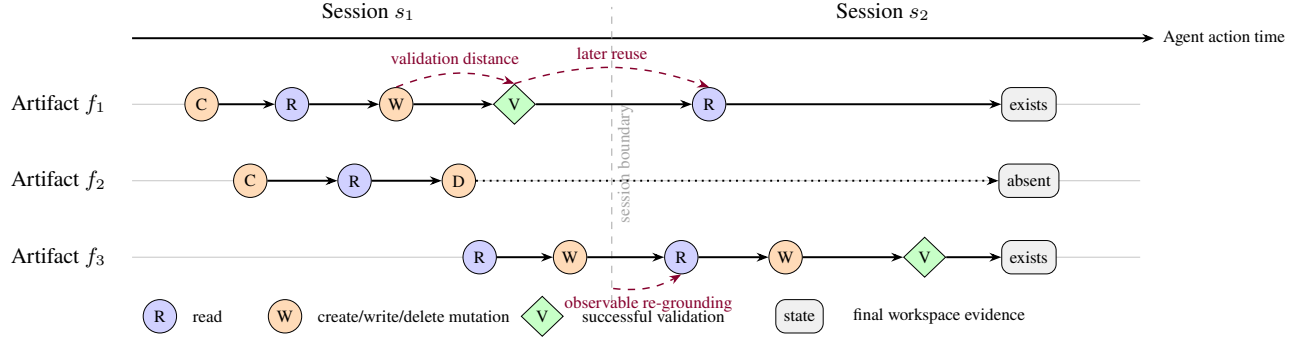


Figure 1: Measurement schematic for persistent workspace evolution. Circles are source-linked reads (R) and mutations (C/W/D); diamonds are successful validations (V); terminal boxes are independently observed final workspace state. Artifact f_1 is mutated, validated, reused after a session boundary, and survives; f_2 is created and later deleted; f_3 illustrates observable re-grounding before continued mutation. The study reports these dimensions separately and never treats the schematic as empirical data.

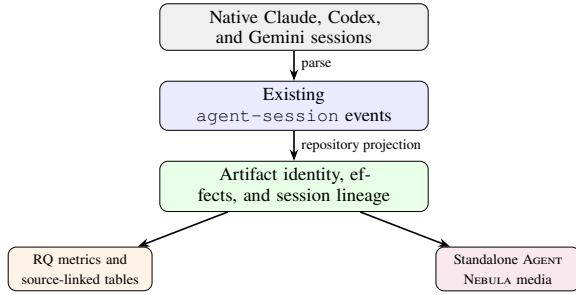


Figure 2: One source projection supports measurement and visualization. The empirical analyses consume structured events, not visual coordinates.

the strongest action-only procedure baseline, and a bounded raw-log model a same-source reconstruction condition.

This design is conditional rather than an executed claim. It requires hashed native prefixes, cutoff workspace state, pinned executable interfaces, and an independent source-explicit oracle before questions or performance values are generated. Section audits those prerequisites and stops the comparison when they are absent.

Implementation

The implementation is a Rust extension of the existing `agentvis` repository projection. It reuses `agent-session` for native parsing, affiliates sessions with all repository worktrees, normalizes paths, and emits ordinary source-linked JSON and CSV rows. The analysis computes project-level coverage, artifact and mutation rows, and RQ summaries without a frontend, database, generated semantic layer, or additional runtime dependency. It reports all admitted Tool actions but uses only actions whose Tool-level workdir or session cwd resolves to a retained worktree for the primary activity-progress comparison.

The product path continues to export one standalone HTML, SVG, PNG, GIF, or MP4 per visualization. Layout is computed once and media outputs share the same frame stream. These outputs provide case evidence navigation and reproducible

examples; the empirical measurements are recomputed from structured rows.

Empirical Study

Research Questions

RQ1—Activity to artifact progress. How much long-running Agent activity becomes artifact change that persists, is later reused, and is followed by successful validation?

RQ2—Validation dynamics. How do successful and failed validations interleave with mutation bursts and worktree-local inter-success mutation accumulation?

RQ3—Repeated-mutation structure. How common and concentrated are first- and repeat-observed mutation episodes across artifacts and Agent action time?

RQ4—Source-session continuity. What continuity is observable between adjacent non-overlapping source-session components, and where does the available source coverage stop interpretation?

RQ5—Workspace activity allocation and migration. How are path-resolved actions allocated among artifact types, and how do their artifact and module transitions and returns evolve in action order?

RQ6—Skill and instruction source coverage. Are visible Skill calls and instruction-file accesses sufficiently complete to support an observational association analysis in this corpus?

RQ7—Matched-comparison readiness. Does the frozen corpus contain the immutable native-log and cutoff-workspace contracts needed to compare Final State, Counts, ProcGrep, bounded raw-log model analysis, and artifact-linked trajectories on the same evidence?

Cases and Inclusion

We fix six local open-source repositories before computing process rates. They cover systems software, mixed documentation and code, autonomous research, and skill/harness development.

Repository	Dominant work type
AgentSight	systems software and research
ActPlane	systems software and research
bpf-developer-tutorial	tutorial, code, documentation
eunomia.dev	content, documentation, software
agentskill-observability-paper	paper, experiment, code
academic-writing-skills	skills, prompts, tests, documents

Table 1: Fixed first-study cases. Each project remains one case; its sessions are not treated as independent projects.

The main corpus includes all repository-direct Claude, Codex, and Gemini sessions and retains actions with no file effect. We report source coverage by project, vendor, session, effect, status, and path presence. Global path-only matches form a separate sensitivity analysis. The frozen corpus contains 2,049 admitted native sessions and 206,249 Tool actions; 1,825 sessions and 175,850 actions resolve to retained worktrees. It yields 7,154 observed artifact identities and 13,152 confirmed mutation rows. All six projects pass the longitudinal gate, although source coverage for individual measurements varies substantially.

Analysis Protocol

Each project is first analyzed as a complete case. Cross-case summaries state their project weighting and do not treat actions as independent samples. Primary outputs are effect sizes, distributions, cumulative/survival curves, and uncertainty. Bootstrap or permutation uses session or project blocks only when exchangeability is defensible. Missing source fields remain coverage results rather than imputed events.

Artifact type is an inspectable path/extension classification with an unknown class. Top-level directories provide the default module unit. Model-derived motifs, embeddings, clusters, or summaries are exploratory secondary analyses and must resolve to source IDs and deterministic metrics.

RQ1: Activity to Artifact Progress

RQ1 reports action and mutation volume, introduced-artifact persistence, later reuse distance, validation-before-supersession distance, competing outcomes, and the conjunction for eligible observation-born introduction episodes. Existing-file writes have unknown content durability unless native diffs, snapshots, or Git line evidence establish it independently.

Figure 3 separates the three dimensions. All six projects support later-reuse analysis: 89.8–97.3% of eligible mutations receive a later observed access, with delete before reuse treated as a competing outcome. However, only AgentSight, ActPlane, and eunomia.dev expose an eligible confirmed create and an adapter-recognized successful validation. The preregistered four-project gate therefore stops cross-case interpretation of persistence and validation; their panels are coverage descriptions, not population estimates. The three observable persistence proportions are 939/983, 3/50, and 4/10, and the corresponding observed validation-before-supersession proportions are 2077/6118, 873/5630, and 32/157.

Figure 4 tests whether raw action volume is an adequate proxy for these outcomes. Only reuse meets the cross-case coverage gate, and its descriptive Spearman rank association with worktree-attributed Tool action volume is $\rho = 0.086$ across six cases. This near-zero six-case rank contrast does not estimate a population effect, but it demonstrates that action count alone does not reproduce the observed reuse ordering in this corpus. RQ1 thus establishes measurable activity–progress separation for reuse while exposing a first-order adapter-coverage limit for persistence and validation.

RQ2: Validation Dynamics

RQ2 measures mutation bursts, recognized successful and failed validation cadence, and worktree-local mutation accumulation between recognized successes. It never calls a successful command proof that every preceding change was covered or correct. Figure 5 shows the exact action-order trajectories and complete inter-success intervals. Only AgentSight, ActPlane, and eunomia.dev expose a recognized successful validation, so the predeclared four-project gate stops a cross-case answer. Within those covered cases, complete intervals are strongly zero-inflated yet occasionally very large: the four eligible worktree lanes have 60.0–89.1% zero-mutation intervals, while their maxima range from 32 to 1,144 mutations. This is evidence about adapter-recognized cadence, not redundant testing or mutation coverage.

RQ3: Repeated-Mutation Structure

RQ3 treats each artifact’s first observed mutation episode separately from later observed episodes. Figure 6 shows that repeat-observed episodes account for 71.8–91.8% of observed mutation episodes across all six qualified cases. Concentration differs substantially: the exact top 10% of mutated identities account for 41.9–86.7% of episodes. These are descriptive properties of the observed trajectories. They do not identify a tail family, convergence, thrashing, defect repair, or wasted work; validation-followed revision and module switching remain open facets of the broader question.

RQ4: Source-Session Continuity

Native records do not provide portable parent/child session roles, and many sessions overlap. RQ4 therefore forms transitive concurrency components within each worktree lane and compares only adjacent non-overlapping components. Figure 7 reports 120 components and 108 boundaries, including mutation-observed prefix composition and defined artifact/module overlap. Fewer than four projects reach 20 eligible boundaries for every estimator; the preregistered gate stops. The figure is thus a source-coverage and within-case continuity audit, not evidence of session reset cost, resumption, model memory, comprehension, or forgetting.

RQ5: Workspace Activity Allocation and Migration

RQ5 analyzes path-resolved activity rather than latent attention or time spent. Figure 8 separates access from mutation and retains both `ok` and `observed` statuses. Status sensitivity is material: for example, eunomia.dev’s mutation allocation to paper/docs is 39.2% with all resolved statuses and 88.2%

with `ok` only. Therefore neither stratum is silently promoted to ground truth.

Figure 9 follows each worktree lane in native call order. Across the six cases, successive path-resolved calls remain on the same artifact 29.3–81.6% of the time, move within the same module 17.5–64.6%, and cross modules 0.9–20.8%. Five cases meet the return-gap gate, with median return gaps of 3–5 calls; the AgentSkill case has only three returns and is N/A. These measurements do not imply duration, internal attention, importance, productivity, entropy, cooling, or forgetting.

RQ6: Skill and Instruction Source Coverage

RQ6 first audits whether the source records expose the configuration needed for an association analysis. We observe exact Skill Tool calls and reads or mutations of `AGENTS.md`, `CLAUDE.md`, and `SKILL.md`. The corpus yields 1,762 signal rows across 1,525 native Tool events, but omits Skill names and arguments, model/configuration fields, repository-external instructions, and proof that a no-signal session was not exposed. Figure 10 therefore records an explicit analysis stop: these visible signals cannot support exposed/unexposed groups or an outcome association, let alone a causal claim.

RQ7: Matched-Comparison Readiness

Before asking which representation answers more questions, RQ7 audits whether every condition can receive the same frozen evidence and an independent oracle. The audit checks six projects against six contracts: a hashed normalized action spine, source linkage, a native admitted-prefix manifest and archive, a cutoff worktree revision manifest, and an explicit untracked-state snapshot. It finds 12 present, zero partial, and 24 N/A project–contract cells. All 206,249 normalized events retain project, vendor, session, event, and source-call linkage, but the frozen study lacks native-prefix archives and per-worktree cutoff state.

Consequently, Counts is admitted only for descriptive prerequisites and the artifact trajectory is coverage-only: its own RQ1 tables cannot serve as an independent accuracy oracle. Final State, pinned ProcGrep, and bounded raw-log model analysis remain N/A. No action-only, artifact-linked, cross-session, or final-state question family passes the planned 30×4 gate. Figure 11 therefore contains no accuracy, coverage advantage, latency, token, or cost value. This is a benchmark-readiness result, not evidence that the proposed representation outperforms a baseline.

Threats to Validity

Native Agent records can omit implicit subprocess effects, external editor changes, or actions performed outside instrumented tools. The study reports adapter and field coverage, and it does not convert missing effects into no-effect observations. Optional system-level AgentSight evidence is a future coverage extension, not a second logic layer in the present analysis.

Final workspace state cannot prove content-level survival of every historical edit. We therefore separate file existence, Git-tracked state, and independently supported content survival.

Successful validation is association rather than coverage or correctness evidence.

The six cases are selected from local author-associated projects. They provide deep, source-native longitudinal evidence but cannot establish population-wide rates. Broad AAI claims require an independent public or prospectively captured corpus after this study identifies measurements with adequate source coverage.

Skill, harness, model, task, and project often change together. More fundamentally, the frozen records do not expose enough fields to identify positive and negative configuration exposure. RQ6 therefore stops at source coverage rather than estimating an observational or causal effect. Likewise, repeated actions, unvisited artifacts, and long validation gaps are process facts rather than automatic failure labels.

Related Work

Agent trajectory studies. Software-engineering trajectory studies analyze action sequences, reasoning, testing, and strategy (Bouzenia and Pradel 2025; Mehtiyev and Assunção 2026). TRAJEVAL identifies coherence collapse after reaching useful intermediate code (Kim et al. 2026). ProcGrep induces canonical action procedures and behavioral fingerprints (Oderinwale 2026). We reuse these action-level precedents and focus the novelty question on persistent artifact identity and cross-session lineage.

Long-horizon and persistent work. AgingBench studies reliability across many sessions (Zhu et al. 2026), while context-management work shows that plans can lose influence (Mehta and Datta 2026). OR-Space evaluates persistent multi-artifact workspaces (Zhou et al. 2026). These works motivate the setting but do not measure introduced-artifact persistence, reuse, validation association, and re-grounding across real project histories.

Diagnosis and harness analysis. AgentDiagnose and AgentRx diagnose completed trajectories (Ou et al. 2025; Barke et al. 2026); TrajAudit retrieves evidence from repository traces (Wang, Xie, and Huo 2026); and HarnessFix links failed trajectories to harness mechanisms (Chen et al. 2026). We do not claim a new diagnostic labeler or causal harness repair. Our tool evaluation asks only which process facts are measurable from source-linked artifact lineage.

Ethical Considerations

Native trajectories can contain private prompts, source code, paths, secrets, and experimental data. Local analysis does not imply permission to publish raw sessions. Released evidence must follow repository and data permissions, remove credentials and personal content, and prefer aggregate or explicitly approved source-linked examples.

Process measurements can be misused as productivity scores. We therefore do not rank people, treat activity volume as progress, or label an observed pattern as waste without independent task evidence. The intended consumer is an Agent or automatic diagnostic process studying its own authorized workspace.

Conclusion

Long-running Agent work persists in artifacts while session context disappears. Studying only final state, commits, or action volume therefore misses the process users need to understand: which work persists, is revisited, receives recognized validation, accumulates repeated mutation, and migrates across a workspace. Across six projects, later reuse is consistently high but weakly ordered by action volume, while repeated mutation is both common and strongly concentrated. Validation, session-continuity, and skill analyses also expose hard source-coverage boundaries rather than portable success, reset, or causal harness effects. Our workspace-centered measurement makes these process facts reproducible while keeping unsupported comparisons explicitly N/A.

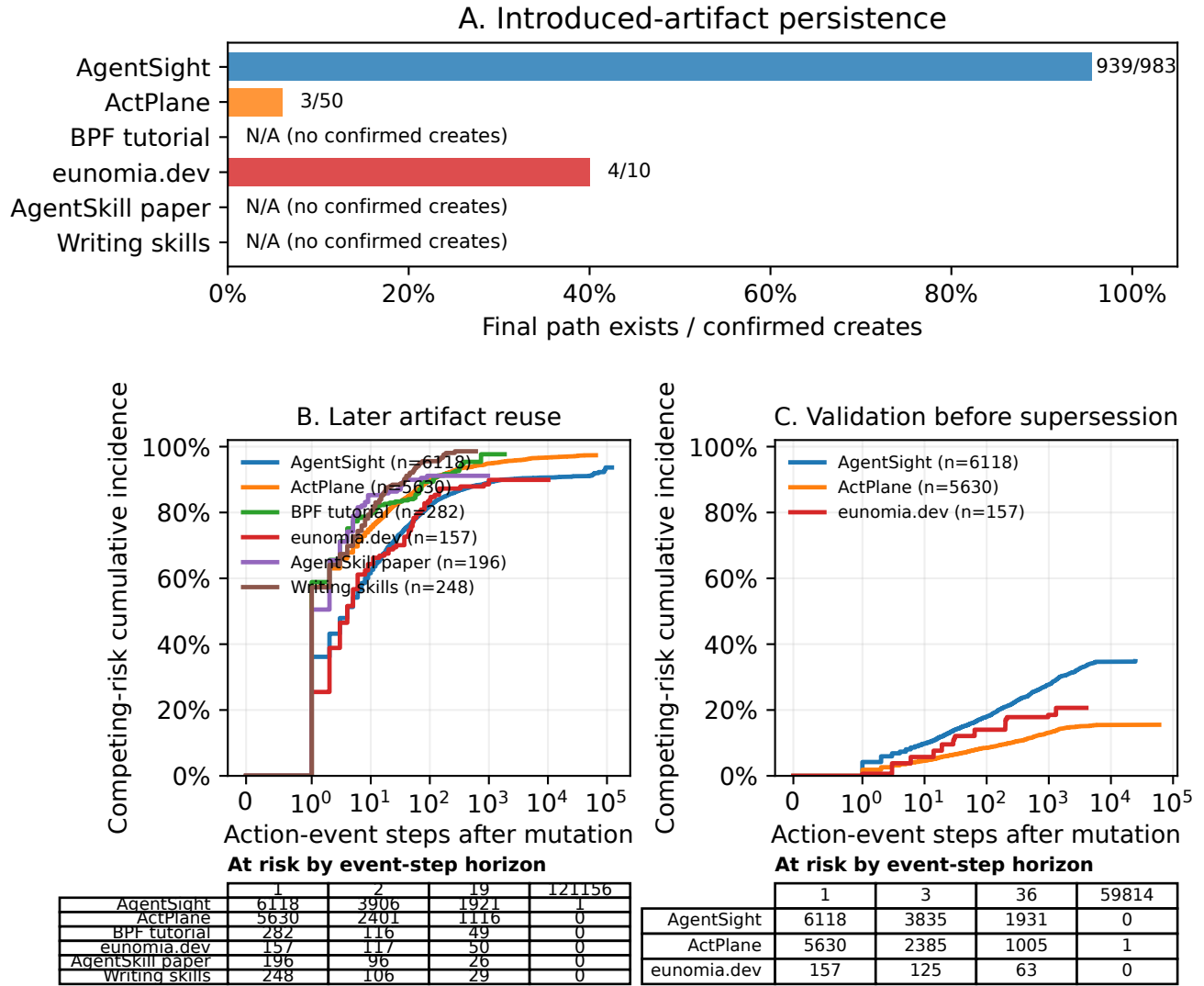


Figure 3: RQ1’s three observable progress dimensions. Panel A reports exact final-path persistence only for adapter-recognized confirmed creates. Panels B and C use Aalen–Johansen cumulative incidence over native Agent action-event distance: deletion competes with later reuse, while the next same-artifact mutation/delete competes with recognized validation. N/A is source ineligibility, not zero progress. Because persistence and validation each cover only 3/6 cases, those panels stop at coverage description.

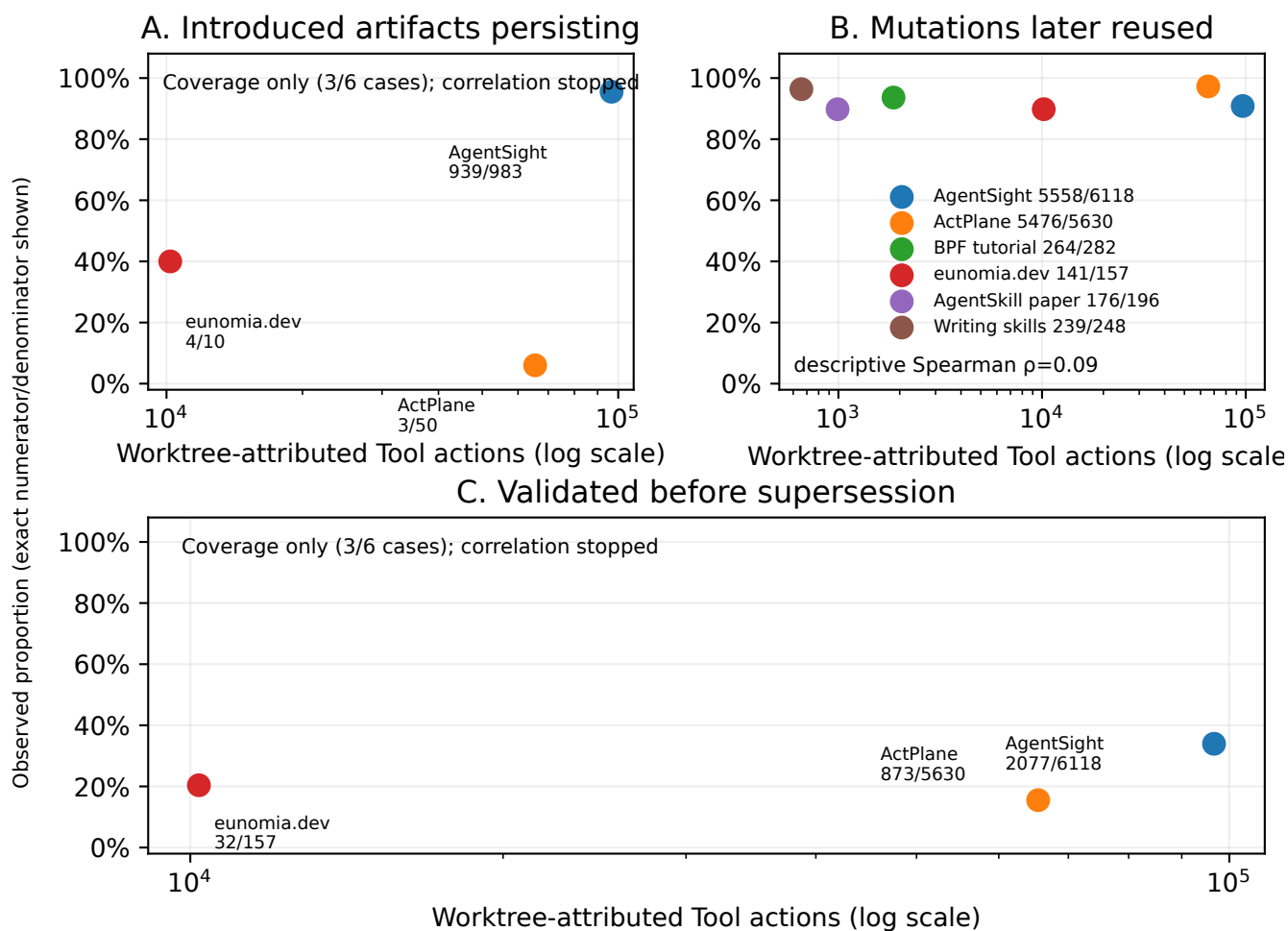


Figure 4: Worktree-attributed Tool action volume versus exact observed RQ1 proportions. Labels show numerator/denominator rather than fitted values. Persistence and validation fail the four-case gate, so no correlation is computed. Reuse includes all six projects and reports only a descriptive rank correlation; actions are not treated as independent cases.

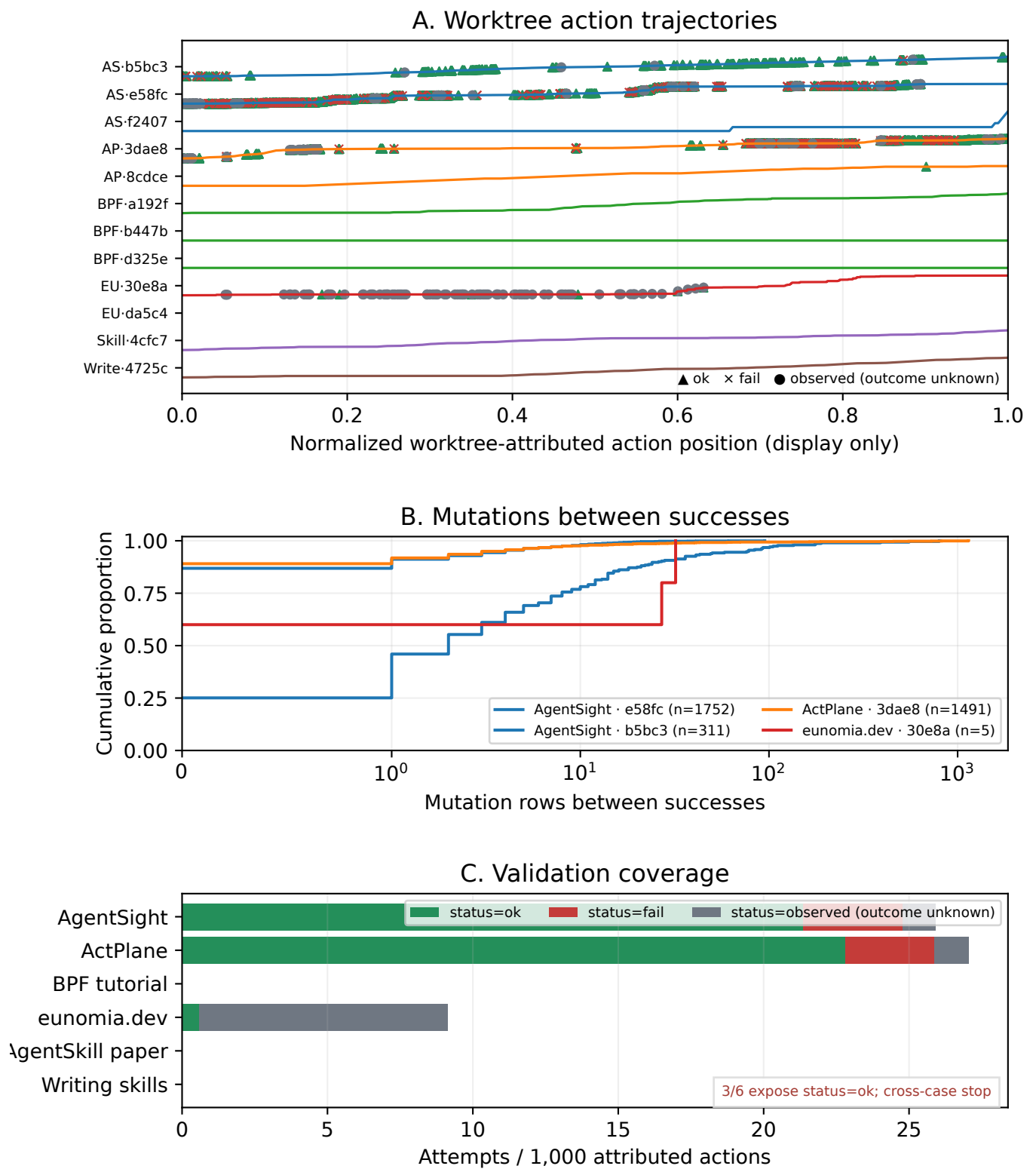


Figure 5: RQ2 recognized-validation dynamics. Panel A follows native Agent action order in every worktree lane; mutation effects project to the affected worktree while validation remains on the command worktree. Panel B reports complete worktree-local intervals between recognized successful validations. Panel C exposes project-level outcome coverage. Only 3/6 cases contain a recognized success, so the analysis stops at coverage and within-case description.

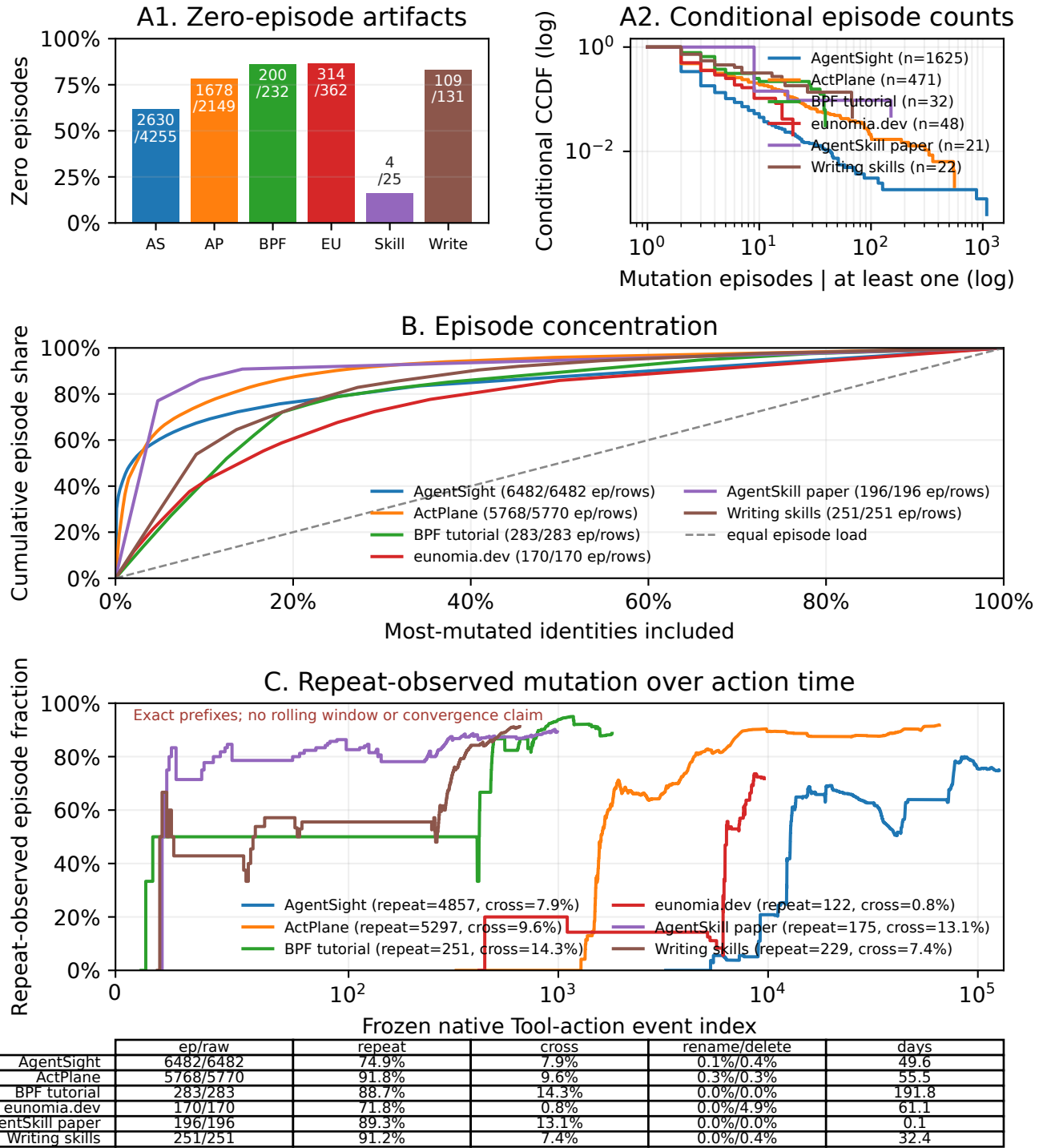
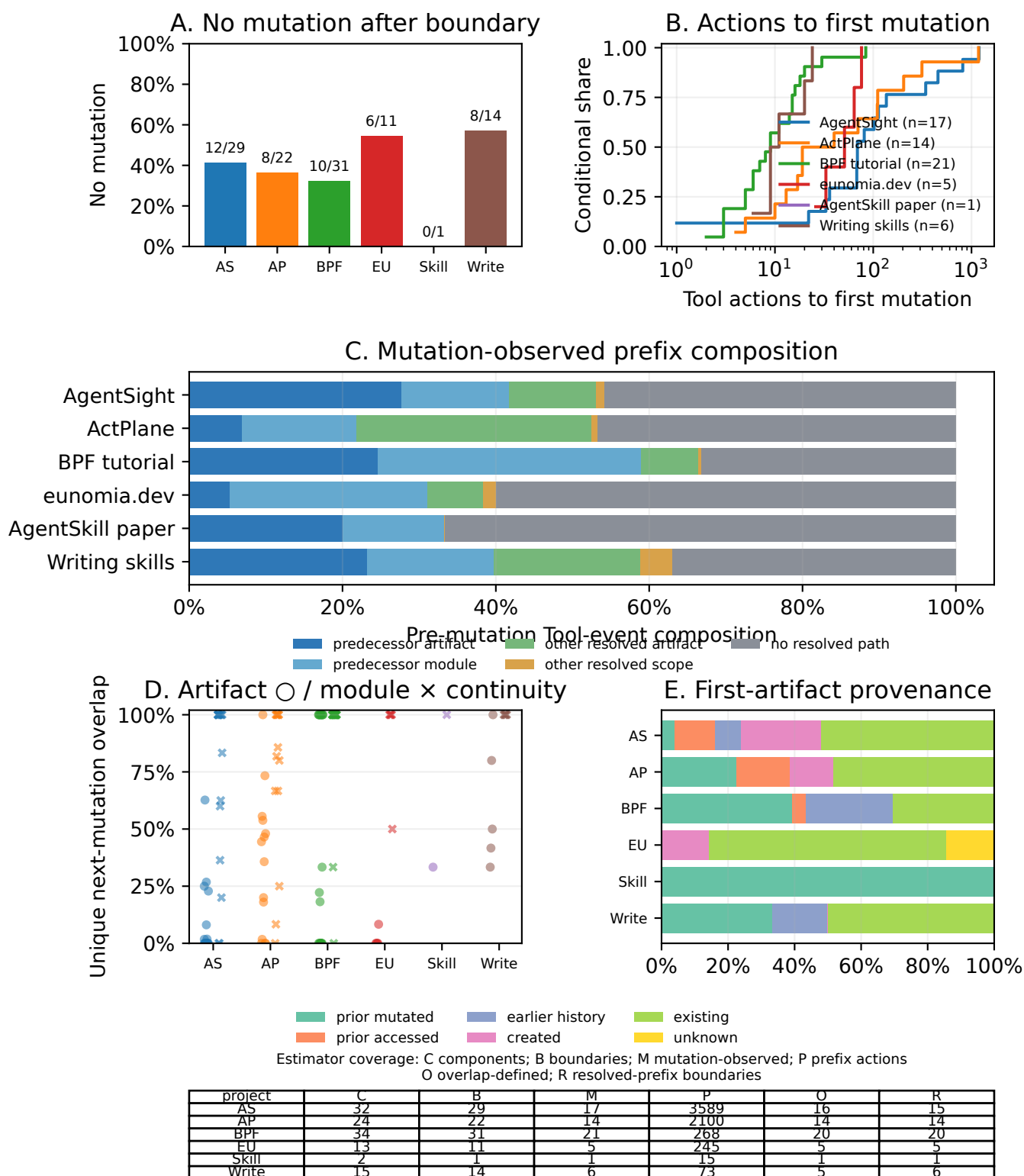


Figure 6: RQ3 repeated-mutation structure for 7,154 observed identities, 2,219 mutated identities, and 13,150 mutation episodes (13,152 source rows). The all-identity zero mass is separated from the conditional CCDF; the concentration curve uses the exact fractional 10% boundary; and the repeat fraction evolves as an action-atomic post-action step function.



All four-project gates stopped: component continuity only; not a reset, resume, memory, or forgetting estimate.

Figure 7: RQ4 source-session component continuity. Adjacent, non-overlapping concurrency components avoid inventing a serial order among overlapping sessions. Timing is conditional on an observed first mutation; empty denominators remain undefined. The stop notice makes the failed four-project gate explicit, so all panels remain within-case descriptions.

RQ5a: Artifact-type allocation and native-status sensitivity

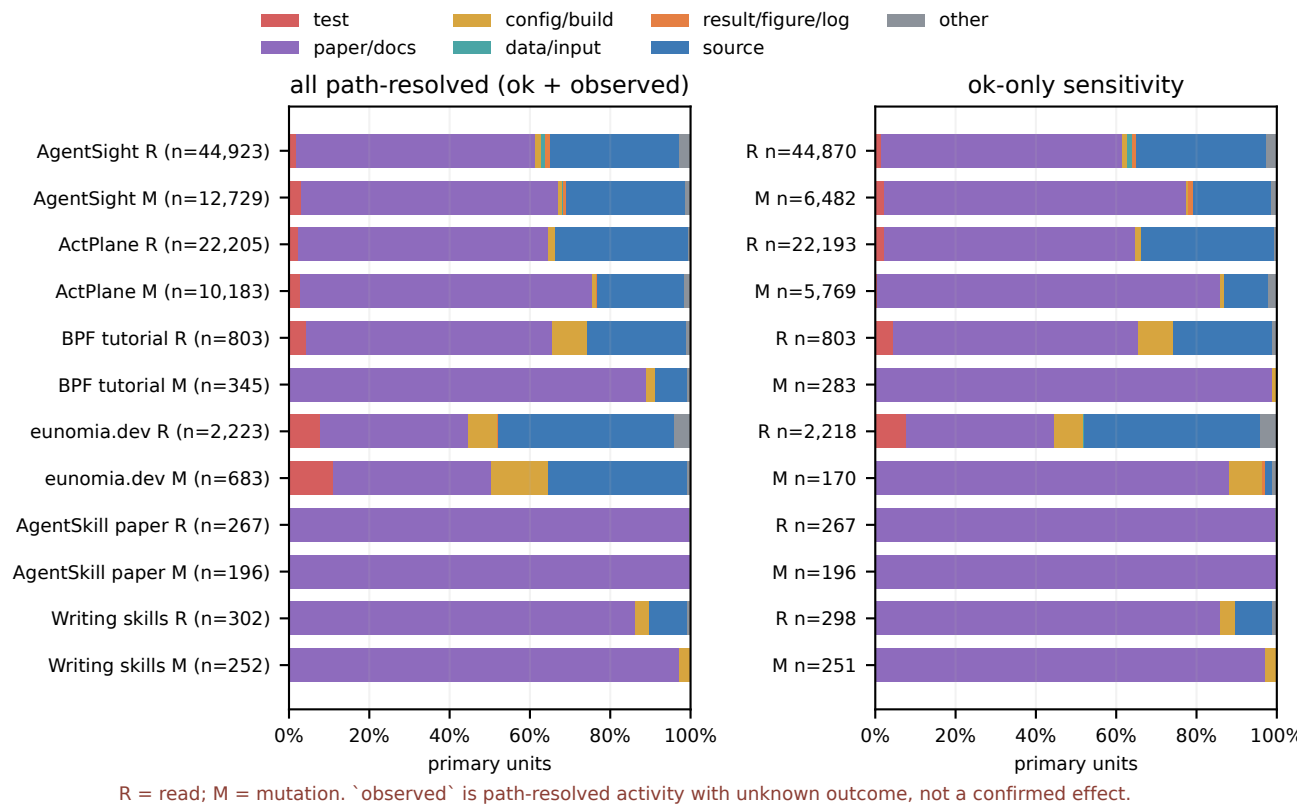


Figure 8: RQ5 path-resolved workspace activity allocation by artifact class, action effect, and source status. One multi-path Tool call contributes fractionally across its resolved paths in the call-weighted view; exact action-weighted rows are retained separately.

RQ5b: Worktree-module activity and source-path migration

A. Top (worktree, module) activity over native action order; row-max color only

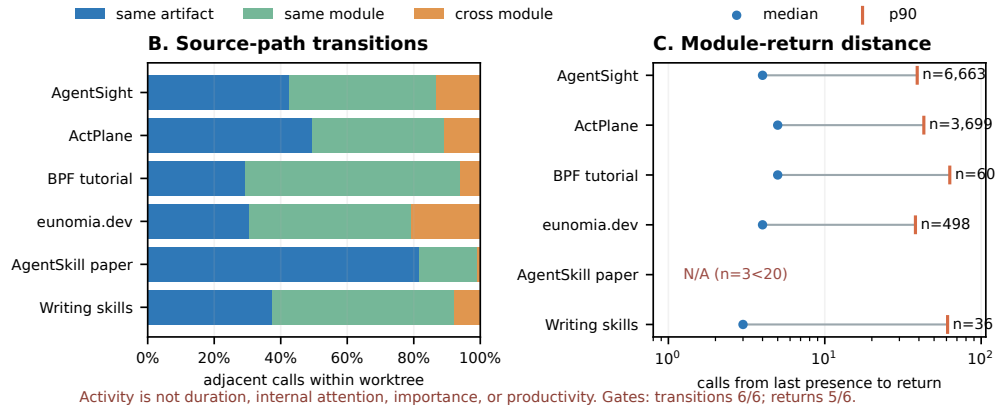
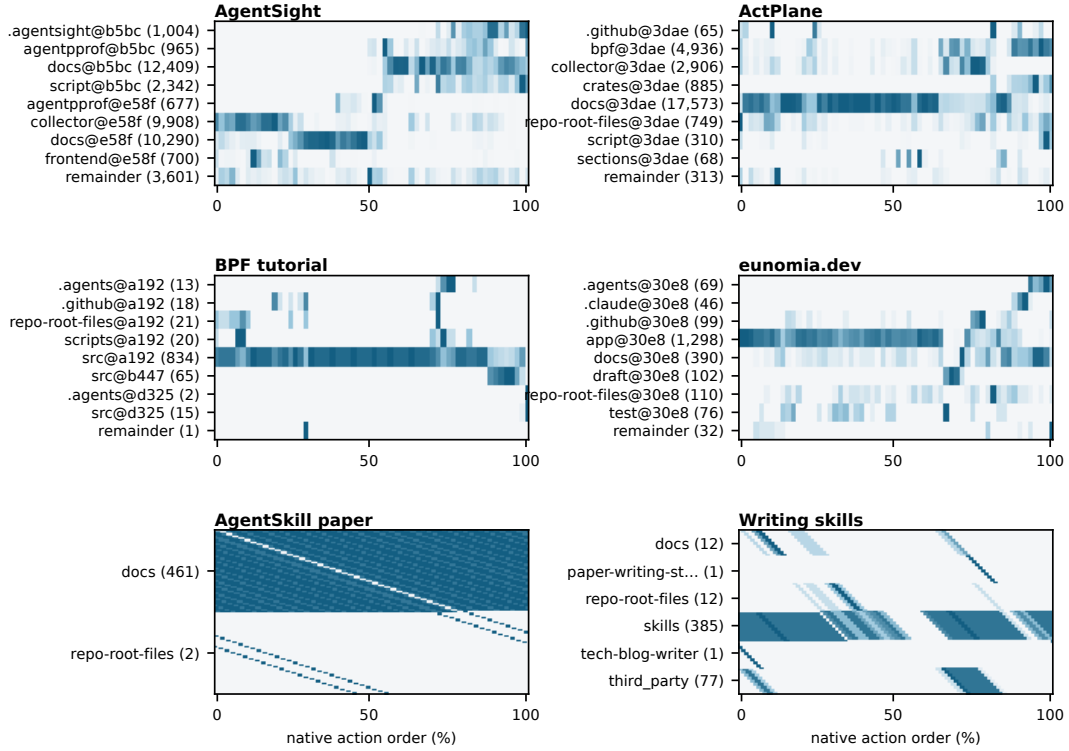


Figure 9: RQ5 workspace activity migration in native call order. Transitions are computed within worktree lanes as same artifact, same module, or cross module; return gaps count intervening calls. Module names and worktree-module keys are reported separately, and low-support estimates remain N/A.

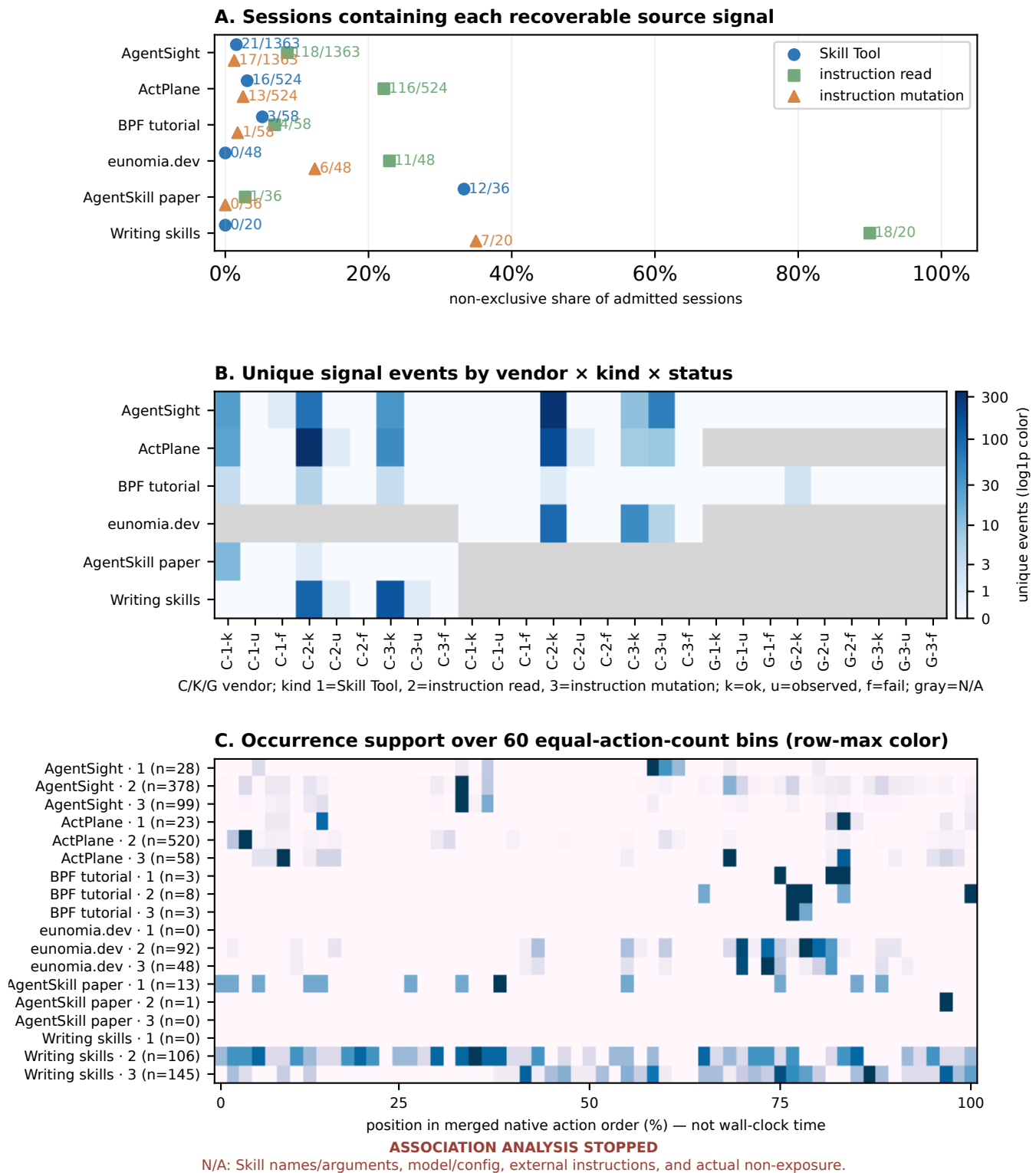
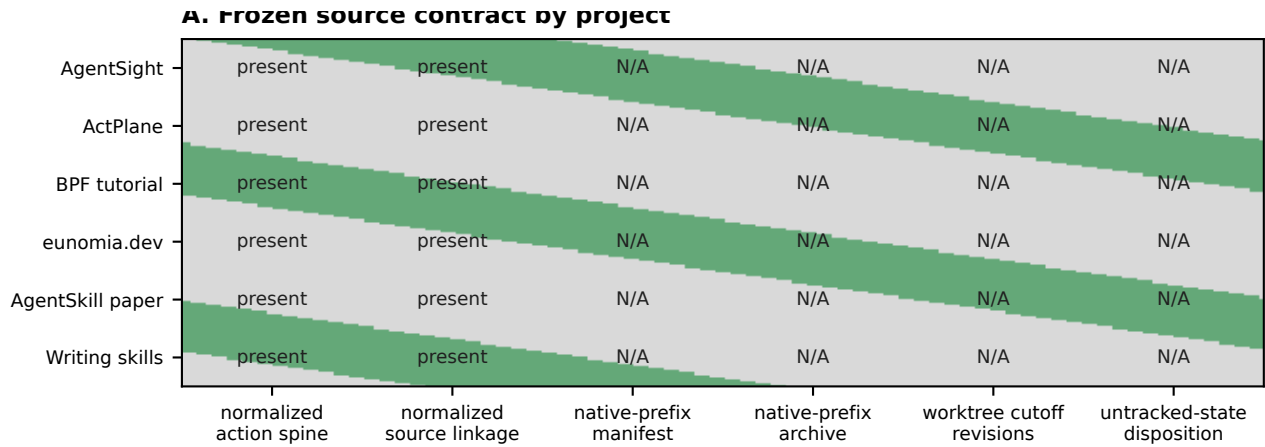


Figure 10: RQ6 source-signal coverage audit. Session shares are nonexclusive; the vendor–signal–status matrix and equal-action-count bins expose where exact visible signals occur. Missing exposure-defining fields trigger the preregistered association stop; N/A is not a null effect.



B. Matched method admission (readiness, not performance)

measured	Counts (normalized)	descriptive counts only; no accuracy oracle
coverage-only	Artifact Trajectory	normalized projection exists; independent matched oracle absent
N/A	Final State	cutoff worktree/untracked contracts missing
N/A	ProcGrep	native prefixes and/or pinned 3-vendor preflight missing
N/A	Raw-log LLM	native prefixes and/or frozen model budget missing

C. Template-family gate

action-only	N/A · 30×4 not evaluated	matched native oracle/ProcGrep/Raw-log unavailable
artifact-linked	N/A · 30×4 not evaluated	no independent source-explicit lineage oracle
cross-session	N/A · 30×4 not evaluated	no frozen native-prefix oracle contract
final-state	N/A · 30×4 not evaluated	cutoff worktree/untracked state unavailable

MATCHED COMPARISON STOPPED

No baseline, oracle, question set, accuracy, advantage, or cost result was produced.

Figure 11: RQ7 source-contract and method-readiness audit. Gray N/A cells denote unavailable matched evidence, not zero performance. The audit stops the comparison before question generation or model execution.

References

- Barke, S.; Goyal, A.; Khare, A.; Singh, A.; Nath, S.; and Bansal, C. 2026. AgentRx: Diagnosing AI Agent Failures from Execution Trajectories. arXiv:2602.02475.
- Bouzenia, I.; and Pradel, M. 2025. Understanding Software Engineering Agents: A Study of Thought-Action-Result Trajectories. In *Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering*.
- Chen, M.; Wang, J.; Liu, Z.; Wang, Y.; Zheng, H.; and Wang, Q. 2026. From Failed Trajectories to Reliable LLM Agents: Diagnosing and Repairing Harness Flaws. arXiv:2606.06324.
- Kim, M.; Wang, D.; Cui, S.; Farmahinifarahani, F.; Zhuo, T. Y.; Garg, S.; Ray, B.; Mukherjee, R.; and Kumar, V. 2026. Coherence Collapse: Diagnosing Why Code Agents Fail After Reaching the Right Code. arXiv:2603.24631.
- Mehta, A.; and Datta, A. 2026. Plans Don't Persist: Why Context Management Is Load Bearing for LLM Agents. arXiv:2606.22953.
- Mehtiyev, T.; and Assunção, W. 2026. Beyond Resolution Rates: Behavioral Drivers of Coding Agent Success and Failure. arXiv:2604.02547.
- Oderinwale, H. 2026. Agent Trajectories as Programs: Fingerprinting and Programming Coding-Agent Behavior. arXiv:2606.16988.
- Ou, T.; Guo, W.; Gandhi, A.; Neubig, G.; and Yue, X. 2025. AgentDiagnose: An Open Toolkit for Diagnosing LLM Agent Trajectories. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP): System Demonstrations*, 207–215.
- Wang, M.; Xie, X.; and Huo, Y. 2026. TrajAudit: Automated Failure Diagnosis for Agentic Coding Systems. arXiv:2605.26563.
- Zhou, C.; Lu, X.; Zhao, J.; Lin, J.; Ge, D.; and Ye, Y. 2026. OR-Space: A Full-Lifecycle Workspace Benchmark for Industrial Optimization Agents. arXiv:2605.28158.
- Zhu, J.; Ro, Y.; Robertson, J. T.; Wang, K.; Li, J.; Vikalo, H.; Akella, A.; and Wang, Z. 2026. Your Agents Are Aging Too: Agent Lifespan Engineering for Deployed Systems. arXiv:2605.26302.